

"ForZero" – Projekt sterownika dla licznika samochodowego wraz z dedykowaną aplikacją umożliwiającą wyświetlanie telemetry w trybie Soft Real-Time

Mikołaj Ratajczak

7 września 2026

Spis treści

1	Wstęp	2
2	Schemat blokowy projektu	2
3	Inżynieria wsteczna licznika	2
3.1	Pinout licznika Audi A4 B6 RB4	2
3.2	Sterowanie kontrolkami	4
3.3	Sterowanie wskaźnikami	4
3.4	Charakterystyki nieliniowych wskaźników	5
4	Projekt sterownika licznika	7
4.1	Schemat blokowy i założenia	7
4.2	Wybór mikrokontrolera	7
4.3	Symulacja rezystancji	8
4.4	Obsługa magistrali CAN	10
4.5	Obsługa kontrolerek	11
4.6	Komunikacja z aplikacją Qt	11
4.7	Projekt PCB	12
4.8	Połączenie zestawu wskaźników ze sterownikiem	13
5	Dedykowana aplikacja w środowisku Qt	14
5.1	Wielowątkowość aplikacji	14
5.2	Symulacja kontrolerek i wskaźników	14
6	Program mikrokontrolera	15
6.1	Środowisko	15
6.2	Pętla zdarzeń	15
7	Testy oraz zabezpieczenia	18
7.1	Zabezpieczenia zastosowane w sterowniku	18
7.2	Opóźnienia	18

1 Wstęp

Celem projektu było stworzenie systemu wbudowanego zdolnego do obsługi wybranych funkcji licznika samochodowego. Jako bazę projektu wybrano zestaw wskaźników z pojazdu Audi A4 B6 o oznaczeniu 8E0920900H RB4, wyposażony w monochromatyczny wyświetlacz FIS. Na obudowie licznika znajduje się oznaczenie daty 05/03/02, co sugeruje, że pochodzi on z marca 2002 roku i należy do początkowego okresu produkcji tego modelu.

Architektura późniejszych rewizji liczników może się różnić, a co za tym idzie – również sposób ich obsługi. Ze względu na te różnice nie jestem w stanie jednoznacznie stwierdzić, czy przygotowana warstwa sprzętowa i programowa będzie w pełni kompatybilna z zestawami wskaźników o innych oznaczeniach.

Ważnym założeniem projektowym było zminimalizowanie kosztu materiałowego urządzenia (BOM) przy zachowaniu założonej funkcjonalności. Przełożyło się to bezpośrednio na decyzje układowe, takie jak zastąpienie droższych scalonych konwerterów poziomów logicznych dyskretnymi układami tranzystorowymi oraz wybór mikrokontrolera wyposażonego w wymagane peryferia.

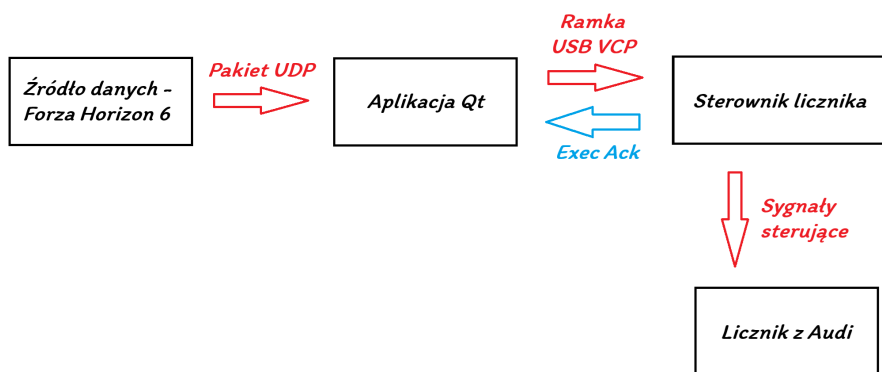
Pliki produkcyjne, lista i kosztorys komponentów, oprogramowanie sterownika oraz autorska aplikacja Qt zostały udostępnione w repozytorium projektu na platformie GitHub: <https://github.com/MikolajDIY/ForZero>.

2 Schemat blokowy projektu

Etap projektowania rozpoczęto od zdefiniowania przepływu informacji między poszczególnymi blokami funkcjonalnymi. Jako źródło danych wykorzystano grę wyścigową *Forza Horizon 6*, która wysyła pakiety telemetryczne za pośrednictwem protokołu UDP. Częstotliwość ich generowania zależy od ustawionego w grze limitu liczby klatek na sekundę.

Pakiety UDP są odbierane przez autorską aplikację utworzoną w środowisku Qt Creator Community 18.0.2. Odbywa się w niej parsowanie danych telemetrycznych oraz symulacja i emulacja wybranych funkcji licznika. Przetworzone dane są następnie przesyłane przez interfejs USB VCP do sterownika, który generuje odpowiednie sygnały elektryczne wymagane przez zestaw wskaźników.

Schemat blokowy projektu forZero



Rysunek 1: Schemat blokowy systemu ForZero

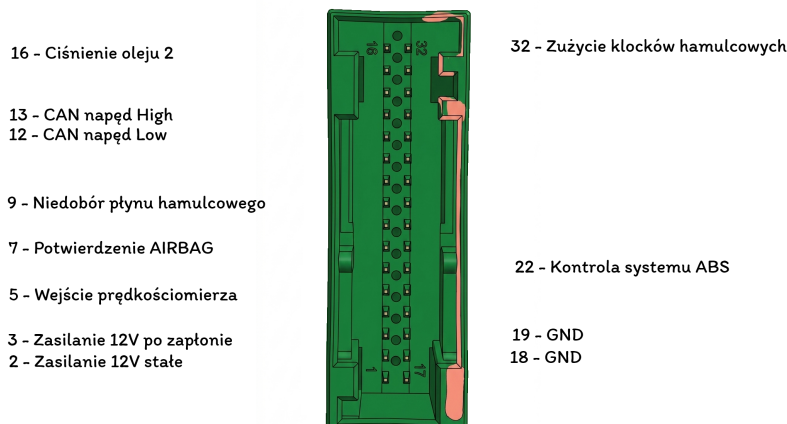
3 Inżynieria wsteczna licznika

3.1 Pinout licznika Audi A4 B6 RB4

Po zdefiniowaniu zakresu projektu przystąpiono do procesu "rozpracowywania licznika". Dużą pomocą było forum *AudiTeam.pl*, na którym znalazłem pełny opis wyprowadzeń licznika Audi A4 B6. Znajdowała się tam również informacja, że zestawy wskaźników z modeli B6 i B7 mają taki sam pinout. Nie udało mi się jednak niezależnie zweryfikować tej informacji.

W pierwszej kolejności należało prawidłowo zasilić licznik oraz znaleźć sposób sterowania wskazaniami prędkości, obrotów silnika, temperatury płynu chłodzącego i poziomu paliwa. Kolejnym krokiem było zidentyfikowanie sygnałów odpowiedzialnych za czerwone i żółte komunikaty ostrzegawcze oraz znalezienie sposobu ich wygaszania.

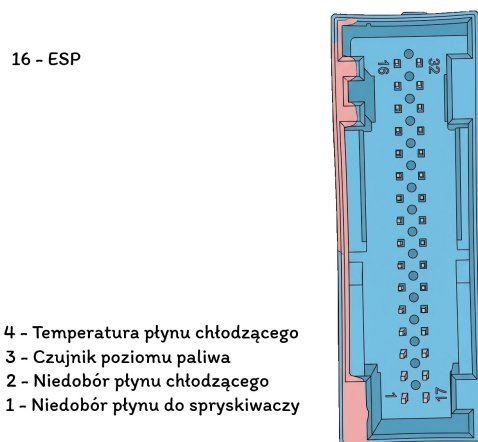
**Wykorzystane piny zielonej
wtyczki licznika Audi A4 B6 RB4**



Na podstawie: <https://www.auditeam.pl/forum/topic/1833-rozpiska-pin%C3%B3w-licznik/>

Rysunek 2: Wykorzystane wyprowadzenia zielonego złącza zestawu wskaźników

**Wykorzystane piny niebieskiej
wtyczki licznika Audi A4 B6 RB4**



Na podstawie: <https://www.auditeam.pl/forum/topic/1833-rozpiska-pin%C3%B3w-licznik/>

Rysunek 3: Wykorzystane wyprowadzenia niebieskiego złącza zestawu wskaźników

Na rysunkach 2 i 3 przedstawiono wyprowadzenia wykorzystane w projekcie. Oprócz sygnałów odpowiedzialnych za sterowanie wskazaniami i wygaszanie komunikatów ostrzegawczych znalazły się wśród nich również wejścia związane z układami ABS oraz ESP. W dalszej części projektu funkcja kontroli trakcji jest opisywana jako TCS. Wybrane źródło danych telemetrycznych nie udostępnia bezpośrednich informacji o interwencji ABS i TCS, ale dostarcza dane dotyczące wciśnięcia hamulca, gazu oraz poślizgu każdego z czterech kół. Na ich podstawie można w przybliżeniu określić moment interwencji tych układów.

3.2 Sterowanie kontrolkami

Nie udało się znaleźć dokumentacji producenta określającej poziomy napięcie dla wejść odpowiadających za wybrane kontrolki i komunikaty ostrzegawcze badanego licznika. W przeprowadzonych testach licznik pracował stabilnie przy sterowaniu tych wejść napięciem 12 V, dlatego wartość tę przyjęto jako roboczy poziom logiczny. Jest to założenie zweryfikowane eksperymentalnie dla używanego egzemplarza, a nie wartość potwierdzona dokumentacją producenta.

Opis sygnałów dla kontrollek

Pin	Opis pinu	Stan logiczny ON	Stan logiczny OFF	Zatrząsk stanu
G7	AIRBAG	N/A	LOW	NIE
G9	Niedobór płynu hamulcowego	LOW	HIGH	TAK
G16	Ciśnienie oleju 2	RPM = 0 LOW RPM > 0 HIGH	RPM = 0 HIGH RPM > 0 LOW	NIE
G22	Interwencja ABS	N/A	LOW	NIE
G32	Zużycie klocków hamulcowych	N/A	LOW	TAK
B1	Niedobór płynu do spryskiwaczy	N/A	LOW	NIE
B2	Niedobór płynu chłodzącego	N/A	LOW	NIE
B16	Interwencja TCS	LOW	HIGH	NIE

N/A - stan domyślny licznika
G - Zielona wtyczka
B - Niebieska wtyczka

Napięcie logiki licznika wynosi 12V,
niższe może nie działać

Rysunek 4: Sposób sterowania wybranymi kontrolkami i komunikatami ostrzegawczymi

Wybrane wejścia licznika posiadają zatrząsk stanu. W przypadku ostrzeżenia o niskim poziomie płynu hamulcowego możliwe jest jego włączanie i wyłączanie podczas pracy licznika. Inaczej zachowuje się ostrzeżenie o zużyciu klocków hamulcowych. Jego wygaszenie wymaga podania odpowiedniego stanu na pin G32 bezpośrednio po podłączeniu licznika do zasilania.

3.3 Sterowanie wskazówkami

Badany zestaw wskaźników można określić jako konstrukcję przejściową, ponieważ wykorzystuje jednocześnie magistralę CAN oraz bezpośrednie sygnały cyfrowe i analogowe. Dobrym przykładem jest sposób sterowania poszczególnymi wskazówkami.

Opis sygnałów wskaźników

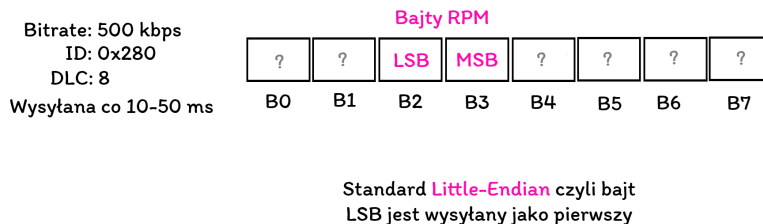
Pin	Opis pinu	Rodzaj sygnału	Typ sygnału	Uwagi
G5	Wejście prędkościomierza	Cyforwy	PWM o zmiennej częstotliwości	1.235 Hz $\approx$ 1 km/h Wypełnienie = 50%
G12	CAN napęd LOW	Cyforwy	CAN	Ramka obrotów wysyłana co 20ms
G13	CAN napęd HIGH	Cyforwy		
B3	Czujnik poziomu paliwa	Analogowy	Symulacja rezystancji	Nie liniowa charakterystyka czujnika
B4	Czujnik temperatury płynu chłodzącego	Analogowy		

G - Zielona wtyczka
B - Niebieska wtyczka

Rysunek 5: Sposób sterowania poszczególnymi wskazaniem zestawu wskaźników

Wskazania poziomu paliwa oraz temperatury płynu chłodzącego zależą od rezystancji czujników podłączonych do odpowiednich wejść licznika. Prędkościomierz jest sterowany za pomocą sygnału impulsowego – wartość wskazania zależy od częstotliwości generowanych impulsów. Obrotomierz jest natomiast sterowany cyfrowo za pośrednictwem magistrali CAN. Częściową strukturę wykorzystywanej ramki przedstawiono na rysunku 6.

Częściowy opis ramki CAN odpowiedzialnej za RPM



'?' - W dniu przygotowywania materiału,
niezidentyfikowane bajty

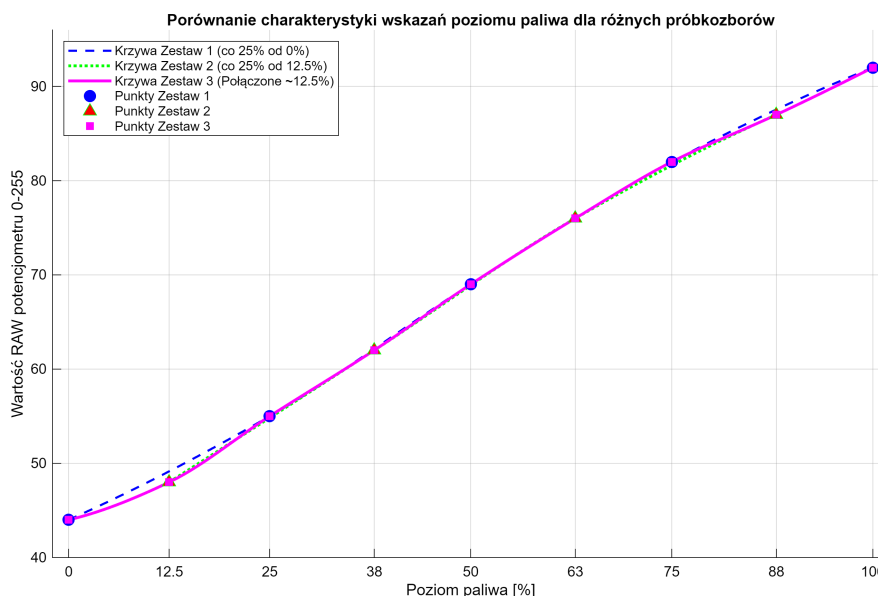
Rysunek 6: Częściowa struktura ramki CAN wykorzystywanej do sterowania obrotomierzem

3.4 Charakterystyki nieliniowych wskaźników

Charakterystyka zależności rezystancji czujnika od wskazywanej wielkości nie musi być liniowa. Dodatkowo na zachowanie wskazówek wpływają wewnętrzne mechanizmy filtracji zastosowane w liczniku. Ich dokładnego sposobu realizacji nie udało się określić.

W przypadku wskaźnika poziomu paliwa filtracja ogranicza szybkie zmiany wskazania, które w samochodzie mogłyby być powodowane przemieszczaniem się paliwa w zbiorniku podczas przyspieszania, hamowania lub pokonywania zakrętów.

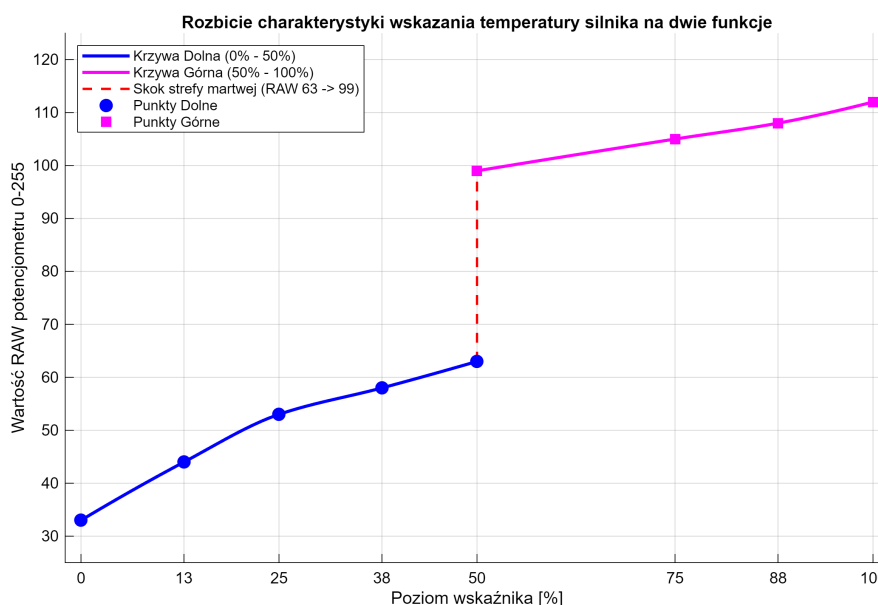
Wskaźnik temperatury płynu chłodzącego również nie odwzorowuje zmian w sposób liniowy. W badanym egzemplarzu dla temperatur z przedziału około 80°C–100°C wskazówka utrzymywała się w pobliżu 90°C. Takie zachowanie ogranicza widoczne wahania wskazania w typowym zakresie temperatury pracy silnika.



Rysunek 7: Zmierzona charakterystyka wskazania poziomu paliwa

Na rysunku 7 przedstawiono zmierzona charakterystykę procentowego wskazania poziomu paliwa w zależności od wartości zadanej potencjometru cyfrowego. Wartości tej nie można bezpośrednio przełożyć na rezystancję widzianą przez licznik ze względu na sposób działania zastosowanego układu emulacji rezystancji. Szczegółowy opis układu znajduje się w sekcji 4.3.

Na wykresie przedstawiono trzy krzywe uzyskane dla różnych zestawów punktów pomiarowych. W kodzie wykorzystano zestaw nr 3, a pozostałe posłużyły do sprawdzenia wpływu doboru punktów pomiarowych na wynik interpolacji. Do interpolacji wykorzystano funkcję PCHIP w środowisku MATLAB. Metoda ta lepiej odwzorowywała zmierzona charakterystykę niż interpolacja wielomianowa używana na wcześniejszym etapie projektu.



Rysunek 8: Zmierzona charakterystyka wskazania temperatury płynu chłodzącego

Na rysunku 8 przedstawiono charakterystykę procentowego wskazania temperatury płynu chłodzącego w zależności od wartości zadanej potencjometru cyfrowego. Temperaturze 50°C przypisano wskazanie 0%, natomiast 130°C odpowiada wskazaniu 100%. Do interpolacji charakterystyki również wykorzystano funkcję PCHIP w środowisku MATLAB.

Należy jednak wspomnieć o istotnym ograniczeniu dotyczącym wskazania poziomu paliwa. Po

uruchomieniu licznika i osiągnięciu prędkości większej od zera zmiana wskazania była możliwa wyłącznie w dół, z tempem około jednej czwartej skali w ciągu 7–10 minut. Nie udało się jednoznacznie ustalić przyczyny ani znaleźć sposobu obejścia tego zachowania.

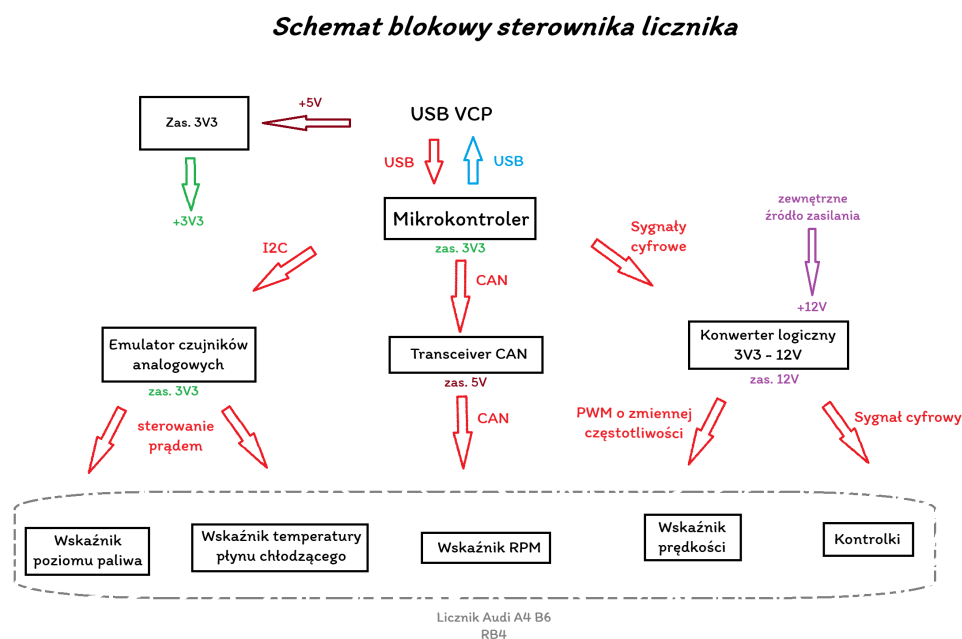
4 Projekt sterownika licznika

4.1 Schemat blokowy i założenia

Podstawą projektowania sterownika było określenie sygnałów wejściowych oraz wymaganych sygnałów wyjściowych. Sposób sterowania poszczególnymi funkcjami zestawu wskaźników opisano w sekcjach 3.2 oraz 3.3.

Do komunikacji pomiędzy aplikacją PC a sterownikiem wykorzystano interfejs USB VCP (*Virtual COM Port*). Pozwala on korzystać z USB jak z wirtualnego portu szeregowego, co upraszcza implementację własnego protokołu warstwy aplikacyjnej. Jednocześnie transmisja korzysta z mechanizmów USB odpowiedzialnych za kontrolę integralności danych i retransmisję pakietów w przypadku wykrycia błędu na magistrali.

Na rysunku 9 przedstawiono schemat blokowy sterownika. Odbiera on ramkę danych z aplikacji PC i przetwarza zawarte w niej informacje na sygnały zrozumiałe dla licznika. Założono pracę w trybie miękkiego czasu rzeczywistego (*Soft Real-Time*). Układ nie jest systemem czasu rzeczywistego z twardymi gwarancjami czasowymi, jednak uzyskiwane opóźnienia są wystarczająco małe dla tego zastosowania. Sposób i wyniki pomiarów opisano w sekcji 7.2.



Rysunek 9: Schemat blokowy sterownika zestawu wskaźników

4.2 Wybór mikrokontrolera

Wybór mikrokontrolera był jednym z ważniejszych kroków projektu. Na podstawie schematu blokowego określono, że układ powinien posiadać:

1. sprzętową obsługę magistrali CAN zgodnej ze standardem CAN 2.0A,
2. co najmniej 20 dostępnych wyprowadzeń GPIO,
3. sprzętową obsługę USB Full-Speed,
4. sprzętową obsługę magistrali I2C,
5. możliwie niski koszt jednostkowy.

Popularne platformy, takie jak **Arduino Uno** czy **Raspberry Pi Pico**, nie spełniały tych wymagań bez zastosowania dodatkowych układów peryferyjnych. Ostatecznie wybrano rodzinę **STM32**, głównie ze względu na szeroki wybór układów z różnymi kombinacjami peryferiów.

Do realizacji projektu wykorzystano mikrokontroler **STM32F105**. Posiada on wymagane interfejsy sprzętowe i umożliwia jednocześnie wykorzystanie USB OTG FS oraz CAN. W tym zastosowaniu nie jest jednak wykluczone użycie innego mikrokontrolera o podobnym zestawie peryferiów.

W trakcie przygotowywania dokumentacji zwróciłem również uwagę na mikrokontroler **STM32C562**, który w sprawdzanym źródle zakupu był około trzykrotnie tańszy od zastosowanego **STM32F105**. Dla obecnej wersji projektu byłby korzystniejszy pod względem kosztu, ale umożliwia obsługę tylko jednej magistrali CAN. Ograniczałoby to możliwość późniejszego rozszerzenia sterownika, np. o dodatkową magistralę CAN Komfort.

4.3 Symulacja rezystancji

Po znalezieniu informacji o rezystancyjnym sposobie sterowania wskazaniem poziomu paliwa oraz temperatury płynu chłodzącego należało określić, jaki zakres rezystancji jest wykorzystywany przez wejścia licznika.

Pomiary rozpoczęto od zmierzenia napięcia pomiędzy wejściami odpowiedzialnymi za poziom paliwa i temperaturę płynu chłodzącego a masą. Uzyskano około 3,5 V. Następnie przeprowadzono próby z rezystorami od 10 kΩ do 47 Ω. Dla około 470 Ω wskazówki znajdowały się na początku skali, dla 100 Ω w jej środkowej części, natomiast przy 47 Ω osiągały pełne wychylenie. Dokładne zakresy rezystancji różnią się w zależności od wskazówki, ale zaprojektowany układ w pełni pokrywa oba zakresy. Pozwoliło to zastosować dla obu wejść dwa bliźniacze układy.

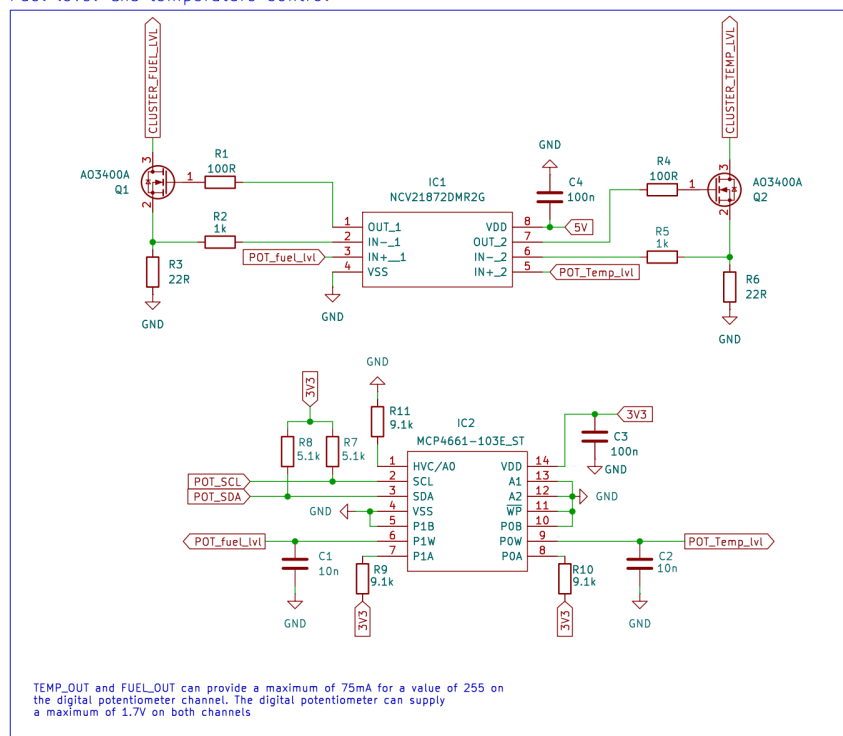
Wymóg symulacji tak niskiej rezystancji nie był dobrą wiadomością, ponieważ wykluczał bezpośrednie zastosowanie typowego potencjometru cyfrowego z dwóch powodów:

1. **Niewystarczająca rozdzielczość w wymaganym zakresie.** Dla potencjometru 10 kΩ o rozdzielczości 8-bitowej zakres odpowiadający kilkudziesięciu omom obejmowałyby zaledwie kilka kroków regulacji.
2. **Zbyt duży wymagany prąd.** Dopuszczalny prąd wielu potencjometrów cyfrowych jest znacznie mniejszy od wartości wynikającej z prostego obliczenia dla 47 Ω. Przy zmierzonym napięciu około 3,5 V otrzymujemy:

$$I = \frac{U}{R} = \frac{3,5 \text{ V}}{47 \text{ } \Omega} \approx 0,0744 \text{ A} = 74,4 \text{ mA}.$$

Jest to wartość orientacyjna, ponieważ rzeczywisty prąd zależy również od wewnętrznej charakterystyki wejścia licznika. Pokazuje jednak, że bezpośrednie użycie potencjometru cyfrowego nie byłoby właściwym rozwiązaniem.

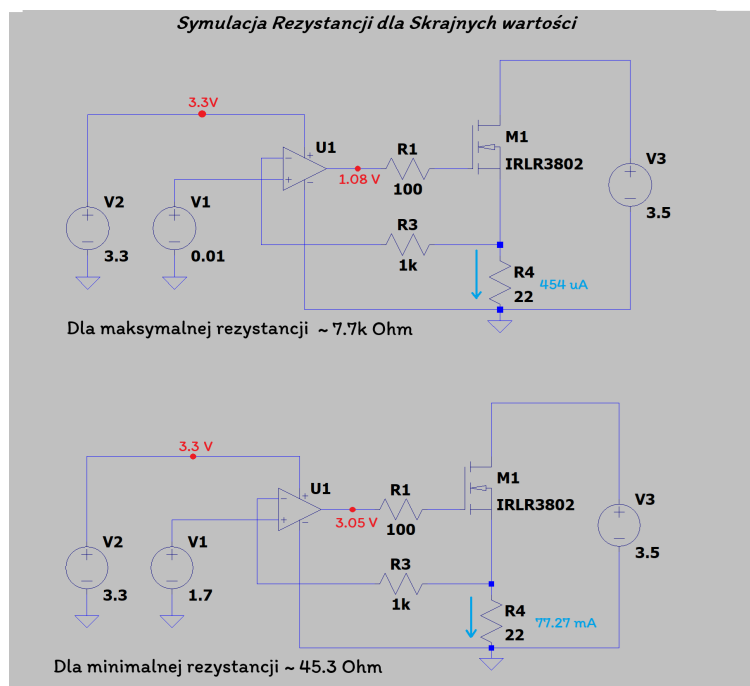
Fuel level and temperature control



Rysunek 10: Schemat układu emulacji rezystancji

Jako element wykonawczy zastosowano tranzystor N-MOSFET typu *Logic-Level* A03400A, którego bramka jest sterowana przez wzmacniacz operacyjny pracujący w pętli sprzężenia zwrotnego. Zastosowany wzmacniacz umożliwia pracę blisko obu szyn zasilania (*Rail-to-Rail*) oraz charakteryzuje się małym dryftem napięcia niezrównoważenia (*Zero-Drift*). Rezystory R3 oraz R6 pełnią funkcję pomiarową dla wzmacniacza operacyjnego oraz ochronną przed zwarcie. Wartość zadaną dla układu ustala cyfrowy potencjometr MCP4661 o rezystancji nominalnej 10 kΩ.

Na rysunku 11 przedstawiono wyniki symulacji wykonanej w programie LTspice. Ze względu na brak odpowiedniego modelu A03400A użyto modelu tranzystora IRLR3802. Parametry obu elementów nie są identyczne, dlatego symulację potraktowano jako wstępną weryfikację koncepcji. Rzeczywistą charakterystykę układu wyznaczono później eksperymentalnie.



Rysunek 11: Wyniki symulacji układu emulacji rezystancji w programie LTspice

4.4 Obsługa magistrali CAN

Magistrala CAN wykorzystuje transmisję różnicową, a napięcie zasilania transceivera nie jest tym samym co poziom logiczny interfejsu mikrokontrolera. Teoretycznie zgodne transceivery powinny współpracować niezależnie od tego, czy po stronie cyfrowej wykorzystują logikę 3,3 V czy 5 V.

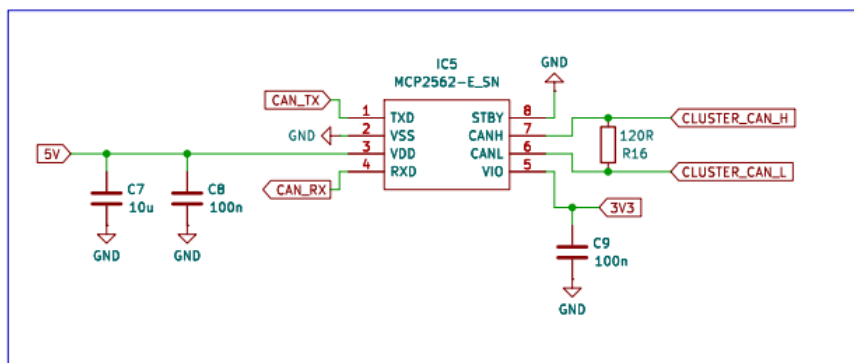
Podczas uruchamiania projektu pojawił się jednak problem. Ramki wysyłane przez gotowy moduł z transceiverem SN65HVD230 były ignorowane przez licznik, natomiast po zastosowaniu modułu z TJA1050 komunikacja zaczęła działać poprawnie. Co ciekawe, w obu modułach znajdował się kontroler MCP2515.

Nie udało się jednoznacznie ustalić przyczyny tej różnicy. Możliwe wyjaśnienia to między innymi:

1. uszkodzenie lub nieprawidłowe działanie używanego modułu z SN65HVD230,
2. różnice konstrukcyjne pomiędzy gotowymi modułami, np. w terminacji lub sposobie prowadzenia sygnałów,
3. różnice parametrów elektrycznych samych transceiverów w konkretnych warunkach pracy.

Na podstawie tych testów nie można więc stwierdzić, że licznik wymaga transceivera CAN zasilanego napięciem 5 V. W finalnym sterowniku zastosowano jednak MCP2562, ponieważ wcześniejsze próby z transceiverem zasilanym z 5 V działały poprawnie.

CAN transceiver



Rysunek 12: Schemat interfejsu magistrali CAN z transceiverem MCP2562

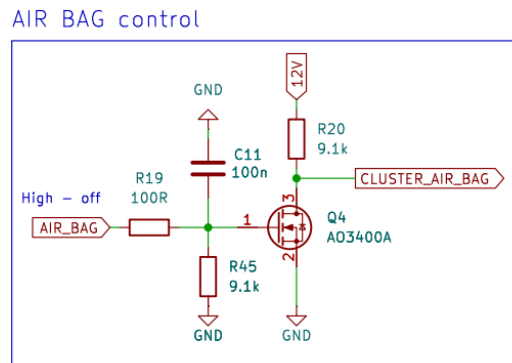
Układ MCP2562 jest zasilany napięciem 5 V, natomiast jego pin V_{IO} pozwala ustawić poziomy logiczny interfejs cyfrowy na 3,3 V. Dzięki temu linie TXD i RXD mogą być połączone bezpośrednio z mikrokontrolerem STM32F105, bez dodatkowego konwertera poziomów logicznych.

4.5 Obsługa kontrolek

Jak opisano w sekcji 3.2, dla wybranych wejść licznika przyjęto roboczy poziom logiczny 12 V na podstawie przeprowadzonych testów.

Wyjścia mikrokontrolera STM32F105 pracują z napięciem 3,3 V, dlatego nie mogą bezpośrednio sterować wejściami wykorzystującymi 12 V. Na rynku nie znalazłem układu scalonego, który przy założonym koszcie i liczbie kanałów pasowałby do tego zastosowania, dlatego konwersję wykonano na tranzystorach N-MOSFET.

Do wykonania nieodwracającego stopnia tranzystorowego potrzebne byłyby dwa elementy aktywne na kanał. Zastosowanie jednego tranzystora N-MOSFET z rezystorem podciągającym do 12 V powoduje odwrócenie stanu logicznego, ale tę inwersję można łatwo uwzględnić w oprogramowaniu. Dzięki temu liczbę tranzystorów dla dziewięciu kanałów ograniczono z 18 do 9, zmniejszając koszt oraz powierzchnię zajmowaną na PCB. Stany generowane przez firmware są z tego powodu odwrócone względem stanów przedstawionych w sekcji 3.2.

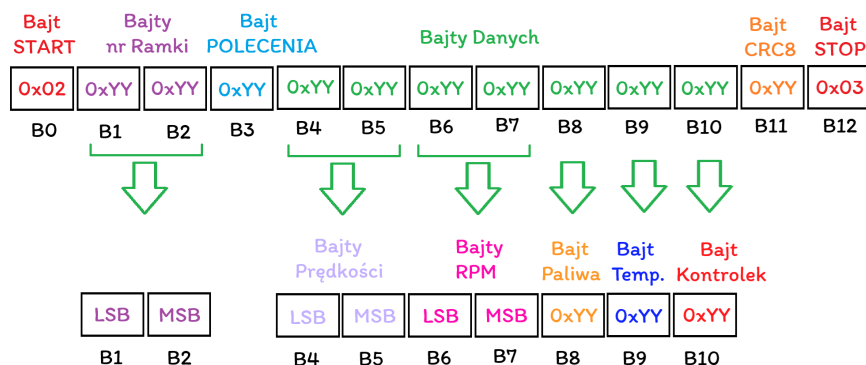


Rysunek 13: Pojedynczy kanał układu sterowania wejściem 12 V

4.6 Komunikacja z aplikacją Qt

Komunikacja sterownika z aplikacją Qt odbywa się przez interfejs USB VCP. USB posiada własne mechanizmy wykrywania błędów i retransmisji, dlatego suma kontrolna CRC8 zastosowana w autorskim protokole nie służy przede wszystkim do wykrywania pojedynczych błędów transmisji na przewodzie. Jest dodatkową kontrolą poprawności kompletnej ramki po stronie mikrokontrolera i pozwala wykryć m.in. sytuacje związane z utratą synchronizacji parsera lub niekompletną ramką.

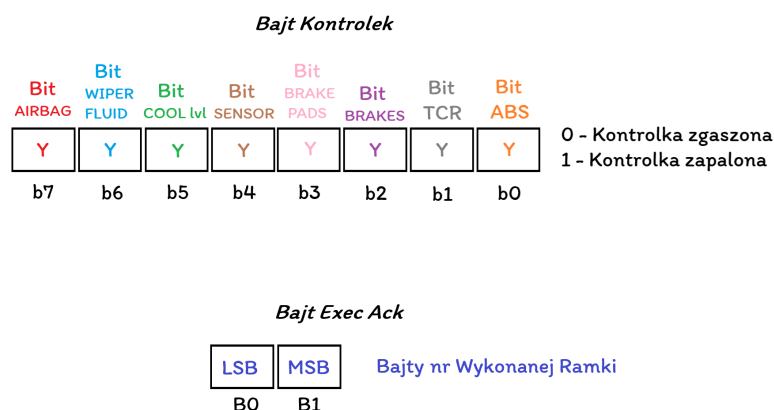
Protokół posiada również limit czasu (*timeout*) wynoszący 50 ms. Jeżeli rozpoczęto odbiór ramki, ale czas od odebrania ostatniego bajtu przekroczy ten limit, indeks bufora jest zerowany. Pozwala to wrócić parserowi do stanu początkowego np. po przerwaniu transmisji lub rozłączeniu urządzenia w trakcie odbioru ramki.

Struktura głównej ramki USB VCP

'0xYY' - Zmienna wartość bajtu od 0 do 255

Rysunek 14: Struktura ramki protokołu komunikacyjnego przesyłanej przez USB VCP

W celu ograniczenia rozmiaru ramki stan ośmiu kontrolek upakowano do jednego bajtu w postaci maski bitowej. Po odebraniu i przetworzeniu nowej ramki sterownik odsyła do aplikacji numer wykonanej paczki jako *Exec Ack*. Pozwala to powiązać potwierdzenie z konkretną wysłaną ramką oraz zmierzyć czas przejścia danych w pętli UDP-PC-sterownik-PC (*round-trip time*, RTT).

Struktura bajtu kontrolek oraz bajtu Exec Ack

'0xYY' - Zmienna wartość bajtu od 0 do 255

'Y' - Zmienny stan bitu 0 lub 1

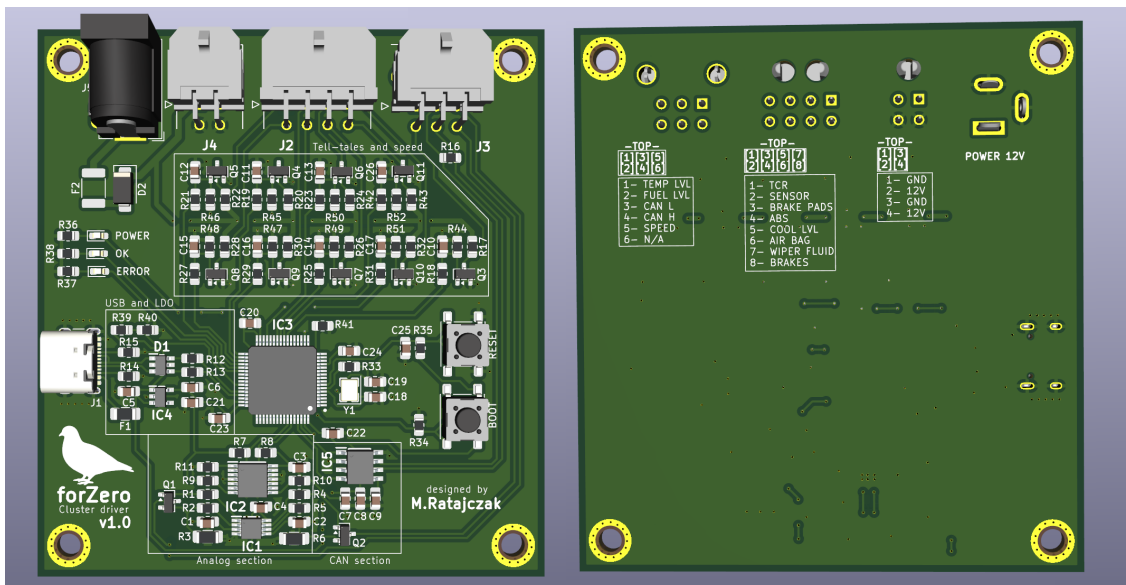
Rysunek 15: Struktura bajtu maski bitowej kontrolek oraz bajtu Exec Ack

4.7 Projekt PCB

Dwuwarstwową płytkę PCB zaprojektowano w programie KiCad. Zastosowano pola miedzi połączone z masą, w tym możliwie ciągłą płaszczyznę masy na dolnej warstwie. W projekcie znalazły się również bezpieczniki polimerowe PPTC na liniach zasilania 5 V i 12 V.

Szczególne uwagę zwrócono na prowadzenie ścieżek sygnałowych. Pary różnicowe USB i CAN prowadzono nad możliwie ciągłym obszarem masy, a linie I2C z dala od potencjalnych źródeł zakłóceń. Jako złącza zastosowano USB-C, gniazdo zasilania typu *DC Barrel Jack* oraz Molex Micro-Fit

3.0.



Rysunek 16: Wizualizacja 3D zaprojektowanej płytki PCB sterownika

4.8 Połączenie zestawu wskaźników ze sterownikiem

Połączenia pomiędzy sterownikiem a zestawem wskaźników zrealizowano zgodnie ze schematem przedstawionym na rysunku 17.

Schemat połączenia STEROWNIK - LICZNIK

Sterownik		Licznik	
Wtyczka:	Nr pinu i opis		Nr pinu i opis
MOLEX Micro-Fit 3.0 x6	1 - TEMP LVL	→	B4 - Czujnik temperatury płynu chłodzącego
	2 - FUEL LVL		B3 - Czujnik poziomu paliwa
	3 - CAN L		G12 - CAN napęd LOW
MOLEX Micro-Fit 3.0 x8	4 - CAN H	→	G13 - CAN napęd HIGH
	5 - SPEED		G5 - Wejście prędkościomierza
	6 - N/A		
MOLEX Micro-Fit 3.0 x4	1 - TCS (w prototypie opisane jako TCR)	→	B16 - Interwencja TCS
	2 - SENSOR		G16 - Ciśnienie oleju 2
	3 - BRAKE PADS		G32 - Zużycie klocków hamulcowych
MOLEX Micro-Fit 3.0 x4	4 - ABS	→	G22 - Interwencja ABS
	5 - COOL LVL		B2 - Niedobór płynu chłodzącego
	6 - AIR BAG		G7 - AIRBAG
MOLEX Micro-Fit 3.0 x4	7 - WIPER FLUID	→	B1 - Niedobór płynu do spryskiwaczy
	8 - BRAKES		G9 - Niedobór płynu hamulcowego
MOLEX Micro-Fit 3.0 x4	1 - GND	→	G19 - GND
	2 - 12V		G2 - +12V
	3 - GND		G18 - GND
MOLEX Micro-Fit 3.0 x4	4 - 12V	→	G3 - +12V

Przyjęta numeracja pinów
we wtyczkach męskich sterownika

TOP	TOP	TOP
[1][3][5]	[1][3][5][7]	[1][3]
[2][4][6]	[2][4][6][8]	[2][4]

G - Zielona wtyczka licznika
B - Niebieska wtyczka licznika

Rysunek 17: Schemat połączenia sterownika z zestawem wskaźników

5 Dedykowana aplikacja w środowisku Qt

5.1 Wielowątkowość aplikacji

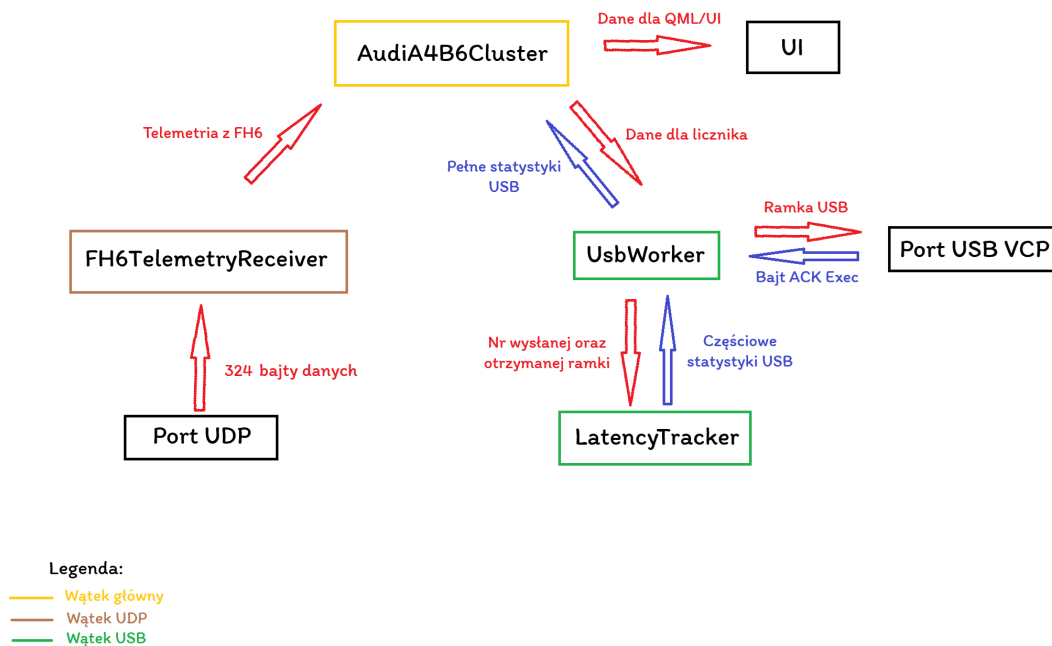
Jako warstwę pośrednią między źródłem danych a sterownikiem wykorzystano autorską aplikację stworzoną w środowisku Qt Creator. Jednym z założeń było możliwie małe obciążenie zasobów komputera. W przeprowadzonych testach aplikacja zużywała maksymalnie około 2% czasu procesora na komputerze z Intel Core i7-11700F oraz około 20 MB pamięci RAM. Są to wyniki dla tej konkretnej konfiguracji i nie stanowią bezpośredniego porównania z innymi aplikacjami telemetrycznymi.

Architektura aplikacji została podzielona na trzy niezależne wątki:

- **wątek główny (GUI)** – odpowiedzialny za obsługę interfejsu graficznego,
- **USB Worker** – odpowiedzialny za asynchroniczną komunikację ze sterownikiem przez USB VCP,
- **UDP Worker** – odpowiedzialny za odbiór i parsowanie przychodzących pakietów telemetrycznych UDP.

Taki podział oddziela operacje komunikacyjne od obsługi interfejsu użytkownika. Dzięki temu odbiór danych UDP i komunikacja przez USB nie blokują wątku GUI, co ogranicza ryzyko jego chwilowego zawieszania podczas intensywnej transmisji.

Schemat blokowy aplikacji Qt



Rysunek 18: Schemat blokowy architektury wielowątkowej aplikacji Qt

5.2 Symulacja kontrolek i wskaźników

Zastosowanie własnej aplikacji umożliwia również symulację załączenia kontrolek TCS, ABS i AIRBAG oraz wskazania temperatury płynu chłodzącego. Przyjęte źródło danych nie udostępnia bezpośrednio informacji o interwencji tych układów ani temperatury płynu chłodzącego, dlatego odpowiednie stany są wyznaczane na podstawie dostępnych danych telemetrycznych. Gra wysyła poziom paliwa, tylko w przypadku włączenia w ustawieniach gry "uszkodzeń symulacyjnych". W przyszłości możliwe jest również dodanie symulacji zużycia klocków hamulcowych.

Warto zaznaczyć, że użyte algorytmy nie są modelami fizycznymi poszczególnych układów pojazdu. Służą jedynie do przybliżonego odwzorowania zachowania kontrolki i wskaźników na podstawie telemetrii dostępnej w grze.

6 Program mikrokontrolera

6.1 Środowisko

Oprogramowanie mikrokontrolera STM32F105 powstało w środowisku STM32CubeIDE z wykorzystaniem bibliotek HAL (*Hardware Abstraction Layer*).

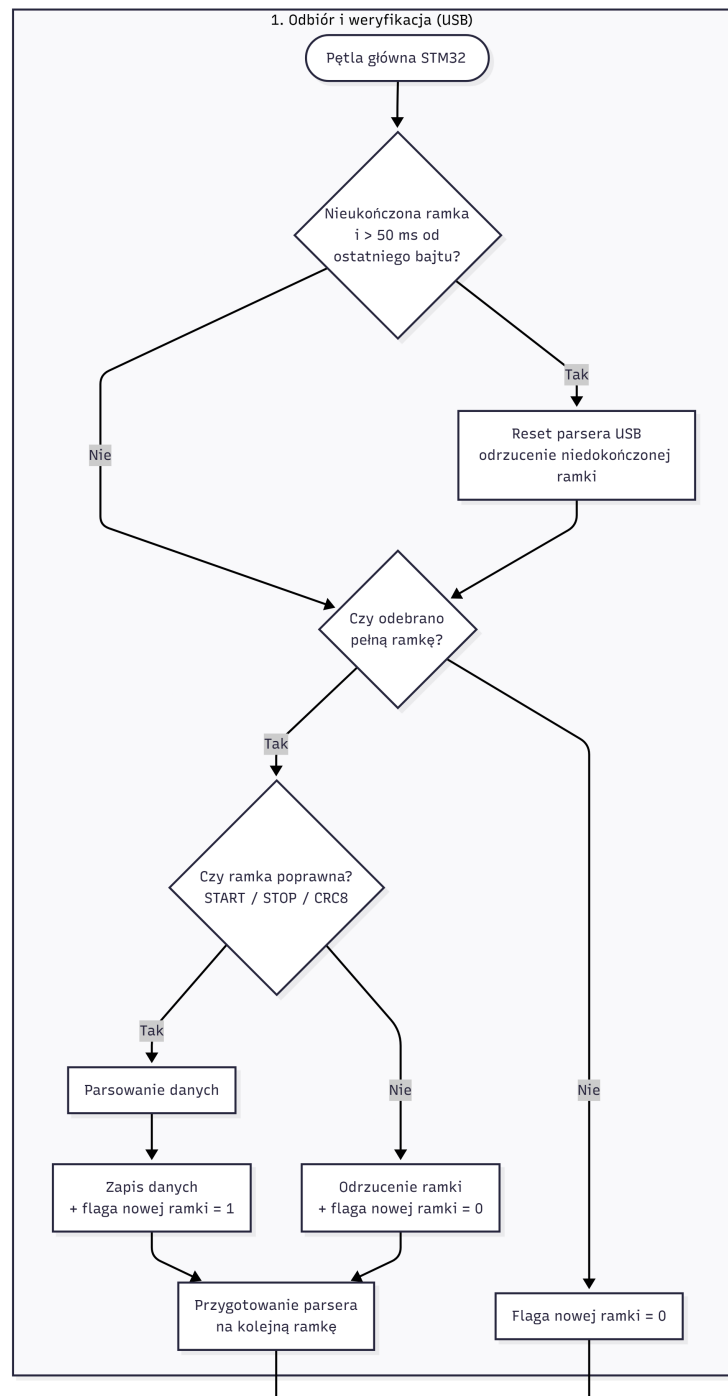
Program można wgrywać do mikrokontrolera przez USB z wykorzystaniem wbudowanego bootloadera. Dzięki temu nie było konieczne dodawanie na PCB osobnego złącza przeznaczonego wyłącznie do programowania. Aby uruchomić odpowiedni tryb rozruchowy, należy nacisnąć RESET, następnie przytrzymując go nacisnąć BOOT, zwolnić RESET, a na końcu BOOT.

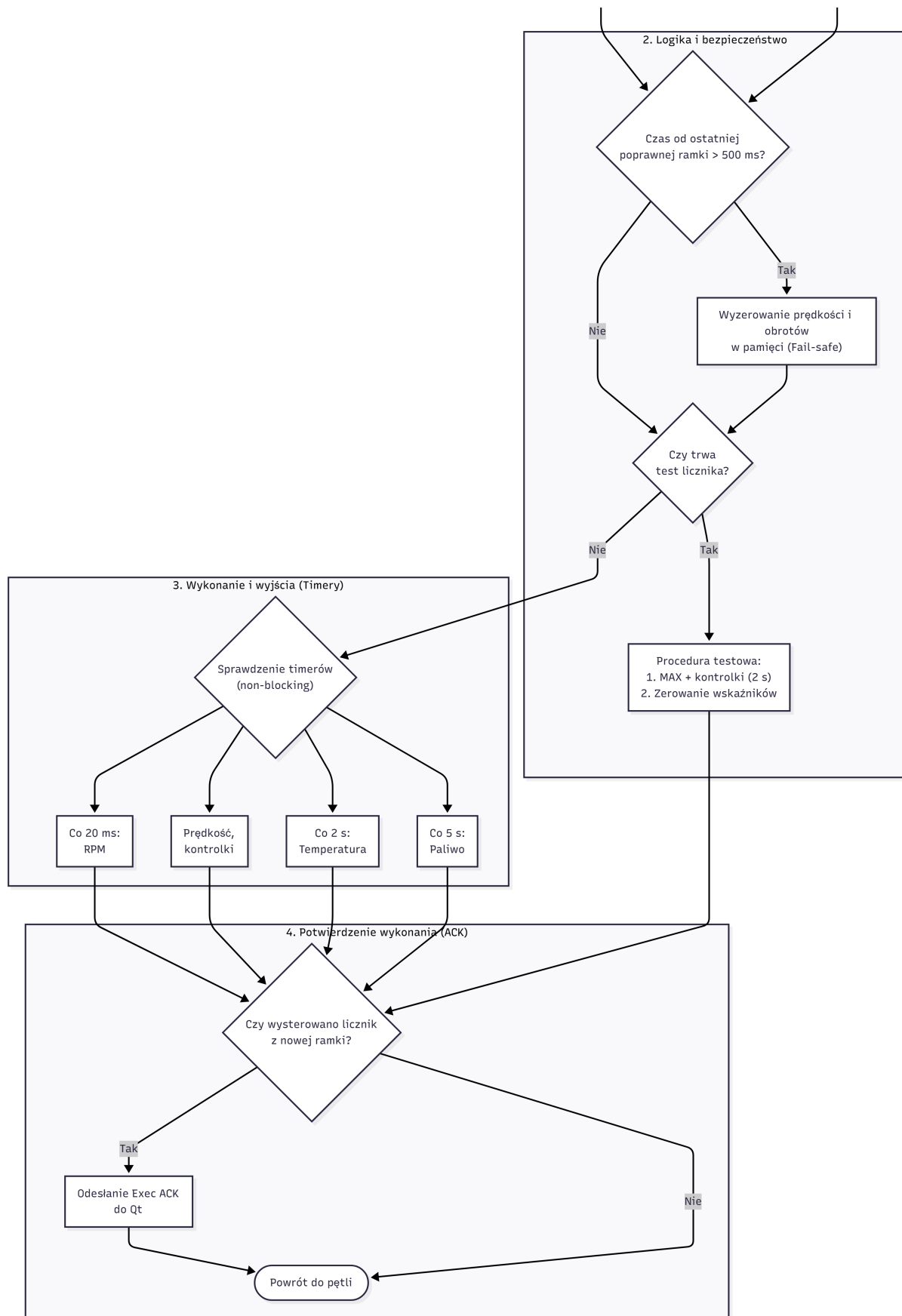
6.2 Pętla zdarzeń

Program mikrokontrolera STM32 opiera się na nieblokującej pętli zdarzeń, z której wyeliminowano funkcje wprowadzające aktywne opóźnienia, takie jak HAL_Delay(). Poszczególne zadania są wykonywane zdarzeniowo lub w określonych interwałach czasowych. Dzięki temu obsługa CAN, I2C i wskaźników nie blokuje pozostałych funkcji programu. Odbiór danych z USB VCP odbywa się asynchronicznie z wykorzystaniem mechanizmu przerwań USB.

Przebieg danych w pętli głównej przedstawiono na rysunku 19. Działanie programu można podzielić na cztery główne etapy:

1. **Odbiór i weryfikacja danych USB.** Po ustawieniu flagi nowej ramki dane są kopiowane do lokalnej struktury local_cluster w sekcji krytycznej. Następnie sprawdzane są bajty początku i końca ramki oraz suma kontrolna CRC8. W przypadku błędu ramka jest odrzucana, a układ pozostaje przy ostatnim poprawnie odebranym stanie.
2. **Logika i zabezpieczenie Fail-safe.** Mikrokontroler kontroluje czas od odebrania ostatniej poprawnej ramki. Jeżeli przekroczy on 500 ms, wartości prędkości i obrotów zostają wyzerowane, dzięki czemu wskazówki wracają do pozycji początkowej. Na tym etapie realizowany jest również test diagnostyczny wskazówek. Jeżeli czas od otrzymania jakiegokolwiek ramki przekroczy 50ms następuje zerowanie indeksu bufora i parser ponownie szuka początku ramki.
3. **Wysterowanie wyjść i obsługa interwałów czasowych.** Odświeżanie poszczególnych sygnałów realizowane jest na podstawie funkcji HAL_GetTick():
 - **prędkość i kontrolki** – aktualizowane są zdarzeniowo, gdy ich wartość zmieni się względem poprzedniego stanu,
 - **prędkość obrotowa silnika (RPM)** – ramka CAN o identyfikatorze 0x280 jest wysyłana co 20 ms,
 - **temperatura płynu chłodzącego** – aktualizowana jest co 2000 ms,
 - **poziom paliwa** – nowa wartość jest przesyłana przez I2C do cyfrowego potencjometru co 5000 ms, jeżeli wcześniej odebrano nowe dane.
4. **Potwierdzenie wykonania (Exec Ack).** Po przetworzeniu nowej ramki i wykonaniu związanych z nią operacji mikrokontroler odsyła do aplikacji Qt komunikat Exec Ack z numerem przetworzonej paczki. Pozwala to powiązać potwierdzenie z wysłaną ramką i wykorzystywane jest m.in. do pomiaru RTT.





Rysunek 19: Schemat pętli zdarzeń oprogramowania mikrokontrolera STM32

7 Testy oraz zabezpieczenia

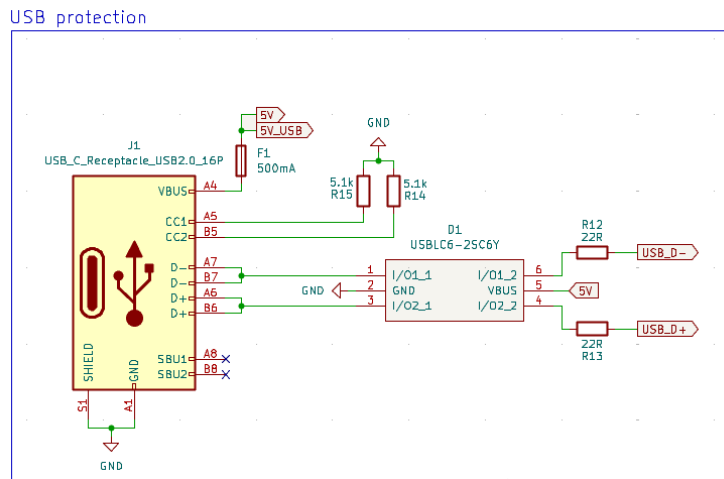
7.1 Zabezpieczenia zastosowane w sterowniku

Sterownik jest jednocześnie podłączony do portu USB komputera i zewnętrznego źródła zasilania 12 V, dlatego na etapie projektowania PCB uwzględniono zabezpieczenia przed zwarcieniem, nadmiernym poborem prądu oraz wyładowaniami elektrostatycznymi ESD.

Linie sygnałowe USB zabezpieczono za pomocą układu USBLC6-2SC6. Rodzina tych układów obejmuje również wariant przeznaczony do zastosowań motoryzacyjnych, kwalifikowany zgodnie z AEC-Q101.

Na linii zasilania USB zastosowano bezpiecznik polimerowy PPTC o prądzie znamionowym 500 mA. Dla zewnętrznego zasilania 12 V zastosowano bezpiecznik PPTC o prądzie znamionowym 1,25 A, dobrany z zapasem względem prądu pobieranego podczas normalnej pracy zestawu wskaźników.

Na rysunku 20 przedstawiono schemat podłączenia gniazda USB-C. Rezystory pull-down R14 i R15 o wartości 5,1 k Ω na liniach CC1 i CC2 pełnią funkcję rezystorów R_d . Pozwalają źródłu USB-C wykryć sterownik jako urządzenie pobierające zasilanie (*Sink*) i pełniące rolę UFP. Zastosowanie rezystorów na obu liniach CC zapewnia poprawną detekcję niezależnie od orientacji wtyczki USB-C.

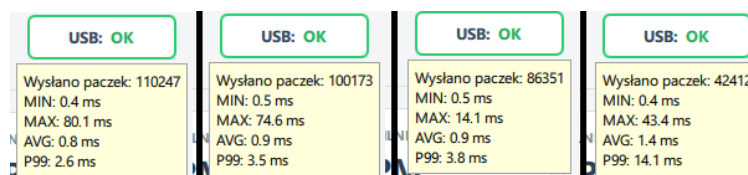


Rysunek 20: Schemat podłączenia portu USB typu C wraz z zabezpieczeniami na PCB sterownika

7.2 Opóźnienia

Pomiar opóźnień w pętli PC $\rightarrow$ sterownik $\rightarrow$ PC (*Round-Trip Time*, RTT) zrealizowano za pomocą komunikatu `Exec Ack`, który zawiera numer wykonanej ramki. Klasa `LatencyTracker` w aplikacji Qt mierzy czas od wysłania ramki do odebrania odpowiadającego jej potwierdzenia i wyznacza statystyki dla 20 000 ostatnich pomiarów. Przy częstotliwości 60 Hz odpowiada to około 333 s, czyli 5 minutom i 33 sekundom danych.

Do przechowywania wyników wykorzystano bufor kołowy (*circular buffer*) o stałej pojemności. Dzięki temu ilość pamięci zajmowana przez historię pomiarów nie rośnie wraz z czasem działania aplikacji.



Rysunek 21: Uzyskane statystyki opóźnień RTT dla różnych zestawów pomiarowych w aplikacji Qt

W większości przeprowadzonych zestawów pomiarowych średnia wartość RTT wynosiła poniżej 1 ms, a 99. percentyl (P99) pozostawał poniżej 4 ms. W najbardziej skrajnym z zarejestrowanych zestawów P99 osiągnął 17 ms. Oznacza to, że 99% zmierzonych czasów przejścia przez pętlę PC–sterownik–PC nie przekroczyło 17 ms.

Dla porównania okres pojedynczej klatki przy 165 Hz wynosi około 6,06 ms, a przy 60 Hz około 16,67 ms. Porównanie ma jedynie charakter orientacyjny, ponieważ RTT obejmuje komunikację i przetwarzanie danych w pętli PC–sterownik–PC, a nie pełny czas od wygenerowania danych przez grę do mechanicznego wychylenia wskazówki. Wyniki pokazują jednak, że opóźnienie wprowadzane przez warstwę komunikacyjną i programową jest niewielkie w stosunku do dynamiki fizycznego zestawu wskaźników.

System można określić jako rozwiązanie miękkiego czasu rzeczywistego (*Soft Real-Time*). Terminowe przetwarzanie danych wpływa na jakość działania, ale sporadyczne przekroczenie założonego czasu obsługi nie powoduje awarii ani sytuacji krytycznej. Uzyskane wyniki RTT są wystarczające dla realizowanej funkcji interfejsu człowiek–maszyna.